本章路線

這章要回答的核心問題是：作業系統如何以標準介面控制異質裝置，同時控制延遲、吞吐、隔離與錯誤？

前置知識。 interrupt 與 DMA、memory mapping、block 與檔案。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

學習目標

- 追蹤 application request 到裝置完成
- 比較 blocking、nonblocking、async 與 buffered I/O
- 定位 I/O bottleneck 而不只看 CPU

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.1 Overview

- 分開追蹤 thread、request、buffer ownership。
- 返回、完成通知、媒體持久化各有條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

I/O 的三條時間線

一次 `read(fd,buf,4096)` 可能只碰 page cache，也可能啟動裝置。必須分開追蹤：①呼叫 thread 的 running/blocked/ready；②request 的 queued/in-flight/completed；③buffer 的 CPU/device 所有權。三者不是同一個狀態機。

假設單一 blocking thread、cache miss、單次 DMA 成功：t=0 進 syscall，t=2 µs 完成檢查並提交；t=3 µs thread 睡眠；t=102 µs DMA 完成；t=104 µs handler 標記完成並喚醒；t=110 µs thread 被排程，t=112 µs 回到 user。裝置服務約 100 µs，呼叫延遲 112 µs，醒來到實際執行另等 6 µs。

不變量： thread 看到成功結果前，資料必須已可由 CPU 讀取；thread 變成 ready 並不表示已返回 syscall。排程、完成通知與資料可見性各有自己的同步條件。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

同一介面，不同操作契約

`fd` 代表已開啟的物件與存取權，不是硬體位址。regular file 可以 seek；pipe/socket 是 byte stream，通常不能 seek；terminal 可能有 line discipline；裝置控制透過型別特定的 `ioctl`。統一入口不保證所有操作都可用。

需要分辨五層「完成」：資料被接受進 user/kernel buffer、driver 提交、controller 完成、媒體持久化、遠端應用處理。`write` 返回 4096 不普遍代表這五層全部成立。持久化要使用檔案系統與裝置都支持的同步契約；網路傳送成功也不能推出收件應用已提交交易。

本章用明確的教學裝置模型推導機制；不把該模型的數字當作某台真機量測。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.2 I/O hardware

- PIO、IRQ、DMA 分別處理命令、通知與搬運。
- 任何取消路徑都不能提前釋放 DMA buffer。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Port、MMIO、controller 與匯流排

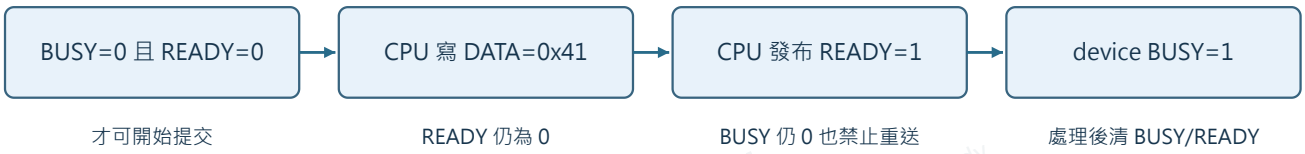
device 是周邊本體；controller 解碼命令、保存狀態並傳資料。CPU 可以用獨立 I/O 指令存取 port，也可透過受保護的虛擬映射讀寫 MMIO register；MMIO 不是一般 RAM，不能任意合併、快取或重排存取。具體順序由架構與 driver 的 I/O accessor/barrier 契約決定。

例：教學 controller 有 STATUS、COMMAND、DATA 三個 register。STATUS.bit0=BUSY，bit1=ERROR；COMMAND.bit0=READY。driver 必須先確認 **BUSY=0 且 READY=0**，再寫 DATA=0x41、設定輸出操作與 READY。若先 READY 再 DATA，裝置可能讀到上一筆值。多個 CPU 提交還需要 driver 的 lock/queue；只有硬體 BUSY bit 不是互斥鎖。

bus 是共用傳輸仲裁點；菊鏈位置與 bus topology 影響存取/中斷路徑，不代表 OS 可以省掉權限檢查。

Polling：完整握手與循環不變量

PIO 輸出 0x41：命令發布晚於資料準備



can_submit = !BUSY && !READY；發布後至完成前，下一筆均不得覆寫 DATA。

Polling：完整握手與循環不變量 （續）

步驟	執行者	BUSY	READY	DATA	可進行的下一步
0	初始	0	0	舊值	driver 可提交
1	CPU 寫資料	0	0	0x41	發布命令
2	CPU 設 READY	0	1	0x41	controller 接收
3	controller 接收	1	1	0x41	輸出資料
4	完成後清命令/狀態	0	0	0x41	CPU 檢查 ERROR

提交的循環前提為 can_submit=(BUSY==0 && READY==0)。READY=1 但 BUSY=0 是「已發布、尚未接受」窗口，仍不得覆寫。單一 producer 也可能在步2立即嘗試第二筆，故只檢BUSY會把DATA=0x41改成0x42，裝置尚未讀便遺失第一筆。

Polling：完整握手與循環不變量 （續）

正確兩筆trace：初始(0,0,舊值)→CPU發布(0,1,0x41)→第二次submit被拒、DATA保持0x41→裝置接受(1,1,0x41)→完成清為(0,0,0x41)→CPU才發布(0,1,0x42)。在單producer、上述兩bit檢查及存取順序保障下，從發布到完成之間can_submit永遠false，故DATA不被下一筆覆寫。若裝置永不完成，polling不會終止；timeout也不表示裝置已停止使用buffer。

Interrupt：事件通知不搬運全部資料

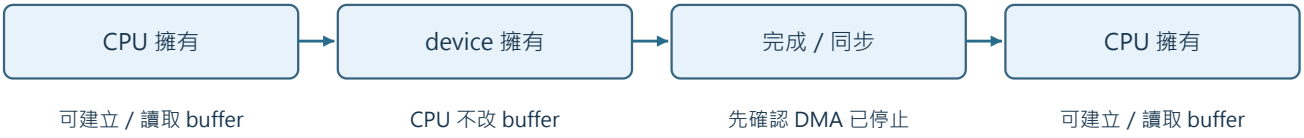
controller 發 IRQ；CPU 在架構允許的邊界保存必要狀態、依 vector 找 handler。handler 判定來源/狀態並 acknowledge，更新 request、排 deferred work 或喚醒 waiter，再返回。共用 IRQ 要檢查「是否本裝置造成」，不能收到 IRQ 就任意完成 queue 首項。

可遮罩的裝置中斷、不可遮罩的 NMI、同步 exception 不是同一分類；不能把整段低 vector 都稱為 NMI。短 handler 與可睡眠的後續處理分開；具體分工依 OS。interrupt 造成 CPU mode/state 切換，只有排程換 thread 時才另有 context switch。

遺失喚醒反例： waiter 先檢查 `done=false`，IRQ 設 true 並 wake，waiter 隨後才睡眠，可能永遠睡。正確流程在同一 lock 下「檢查條件→入 wait queue→釋放 lock 並睡眠」，醒後以 while 重查；IRQ 在同一同步規則下發布 done。通知不是可累積的完成證據，done predicate 才是。

DMA：地址、資料一致性與 buffer 壽命

4096 B 接收：地址映射與 ownership 不能省略



submit 前：map / cache 維護 → descriptor / ownership → doorbell

terminal 後：完成必要同步 → 發布結果 → unmap / 釋放（僅在裝置已不再存取後）

CPU 建 descriptor `{device_addr, dma_addr, length=4096, direction=FROM_DEVICE}`，準備映射/必要 cache 維護後發布；controller 搬資料，最後以 completion 通知。DMA address 可以經 IOMMU 轉譯，不必等於 user VA 或直接等於 physical address。IOMMU 限定 device 可 DMA 的範圍，不能取代 fd 存取權檢查。

DMA：地址、資料一致性與 buffer 壽命 （續）

本例 CPU→device 轉移 ownership 後，CPU 不讀寫接收區；完成且完成必要同步後才讀取。CPU 必須保留 descriptor、映射與 backing pages，直到確知裝置不再存取。釋放 user VA 不代表 DMA 自動取消。非 coherent 平台尤其要使用指定 DMA API；CPU memory barrier 不會自動完成 cache invalidate。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

PIO、interrupt、DMA 成本分開算

教學模型：4 KiB，PIO 每次搬 4 B，CPU 每次 80 ns，共 1024 次，因此資料搬運 CPU 時間 81.92 µs。改 DMA，提交 2 µs，IRQ/收尾 1 µs，CPU 成本 3 µs；裝置傳輸仍需 100 µs，並非 3 µs 完成。

若忙等每次檢查 100 ns、每 5 µs 檢查一次，100 µs 服務期內最多約 20 次、成本 2 µs，但完成偵測另有 0–5 µs 延遲（均勻相位假設下平均 2.5 µs）。每完成一次 IRQ 成本 1 µs 時，2000 次/s 的 IRQ CPU 佔用為 0.002 s/s=0.2%；固定 5 µs polling 卻需每秒 200000 次、2%。

DMA 仍消耗 memory/bus bandwidth；CPU 可並行計算不表示傳輸沒有系統成本。小筆傳輸可能由 PIO 較省 setup；是否較快必須固定資料量、延遲與 CPU 分母比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.3 Application I/O interface

- block/character/network 的操作集合不同。
- short I/O、EOF、EAGAIN 與 async completion 明確分支。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Block、character、network 與 memory mapping

block device 按 block 定位、可支援 random access；character device 提供序列資料，但介面是否保留 record、能否 seek 都依裝置契約。file `mmap` 把檔案內容映進位址空間，由 page fault/page cache 處理；controller MMIO 是寄存器存取，不能混同。

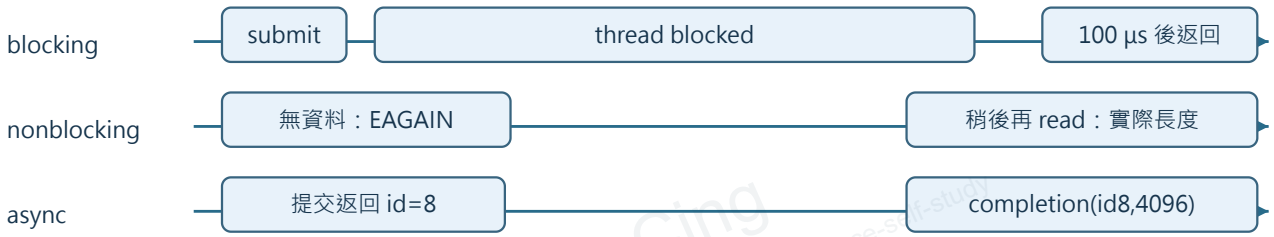
raw/direct I/O 旨在繞過部分快取/檔案層服務，仍受權限、buffer alignment、生命週期與隔離限制。network socket 分出 endpoint、連線與傳輸；stream 不保留 write 邊界，datagram 則有自己的 message/truncation 契約。readiness API 告知可嘗試操作，不替應用搬資料，也不保證另一個 thread 沒先消耗資料。

timer 裝置提供時鐘、中斷與計時。虛擬 timer 可用按 deadline 排序的 queue：在 now=7 ms 時存在 deadline 10、13、13 ms，硬體只設 10；到期取出 10，再設 13，到 13 時取出兩項。精確同時性不保證 callback 當下立即執行。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Blocking、nonblocking、asynchronous 三種觀察

同一讀取，三種 API 契約（不是三種搬運硬體）



device 的服務時間相同；差別是呼叫控制流與結果通知時點。

blocking：呼叫可睡眠，結果返回時代表該次操作結束，但仍可能 short read。nonblocking：沒有資料時立即回 EAGAIN；先 poll/select 再 read 仍須處理競爭後的 EAGAIN。asynchronous：提交返回與完成分離，完成項包含 id、status、actual length；非成功提交可能根本沒有 request。

Blocking、nonblocking、asynchronous 三種觀察 （續）

stream 有資料 200 B 而要求 512 B，合法結果是 200；更新 `done=200`、`remaining=312`、`dst=buf+200`。下一次回 112 則 done=312、remaining=200；再回 0 代表 EOF，應把 312 B 的短結果交給上層或回報協定截斷。回傳 -1 時不把負值加進 done；EINTR/EAGAIN 的處置依 API 與已完成進度。

向量 I/O：部分結果如何跨 iovec

`writev` 有三段長度 [100,200,300]，總計 600 B。若此次成功傳 250 B，先完整消耗第 0 段 100，再消耗第 1 段 150；重試向量應是 `[base1+150,50]`、`[base2,300]`，共 350 B。再次從原向量重送會重複前 250 B。

向量介面減少組成連續暫存區的 copy 與 syscall 次數，仍不保證裝置零 copy，也不保證跨所有檔案/網路類型的任意長度原子性。原子寫入、寫入順序與耐久性是三個獨立規格。

延遲非同步 I/O 使用的 user buffers 必須活到 completion；若改用內核 copy，成本和生命週期改變。取消只要求停止尚可停止的工作；API 說取消失敗/已完成時，不能立刻釋放仍可能被裝置使用的記憶體。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.4 Kernel I/O subsystem

- buffer/cache/spool 分開建模。
- 排程、錯誤、權限與 kernel dispatch 共同約束 I/O。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

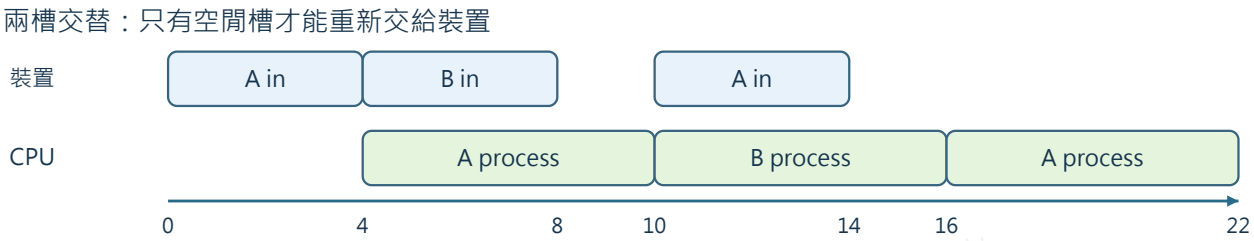
Kernel I/O：排程、buffer、cache、spool 各解什麼問題

scheduling 決定待服務 requests 的次序；buffering 吸收速率/單位不匹配並保存 copy semantics；caching 保存可重用資料；spooling 將完整工作排入較獨占的裝置 queue。單一記憶體區可兼具 buffer/cache 角色，但目的與正確性條件不同。

三個 read 依序到達 block 40、10、41；若排程改成 10→40→41，可減少某些 HDD 移動，但不得越過必要的依賴/barrier，也不能把這套 seek 模型當作 SSD 機械延遲。公平性、deadline 與吞吐可能衝突。

write-back cache 的 dirty copy 可能是最新資料唯一副本；「cache 一定只是別處已有相同資料」在 dirty 狀態不成立。buffered write 已接受、device write 已完成、durable 都不能混為一談。

Double buffering：明確時序與可用槽位



ms；每塊輸入 4 ms、CPU 6 ms。B 在 8–10 ms 等 CPU；A 在 14–16 ms 等 CPU。

0–4 填滿；4–16 重疊；16–22 排空。3 塊完成於 22 ms，並非 18 ms。

每塊裝置輸入 4 ms、CPU 處理 6 ms；單 buffer 每塊先輸入後處理，3 塊共 30 ms。雙 buffer：A 輸入 0–4、處理 4–10；B 輸入 4–8、等 8–10、處理 10–16；A 再輸入 10–14、等 14–16、處理 16–22，3 塊共 22 ms。穩態週期仍是 max(4,6)=6 ms。

4–8 ms 兩單元可重疊，但到 8 ms 不能覆寫仍由 CPU 使用的 A。對 N 塊、可完全重疊且兩槽足夠的本模型，總時長 t_in+t_cpu+(N−1)max(t_in,t_cpu)。初始化/排空不能從有限 N 的公式中刪掉。

Copy semantics、spooling 與錯誤界線

copy semantics：應用寫入 buffer="ABC" 後立刻改成 "XYZ"，若 write 契約在返回前已複製/完成讀取，後續輸出仍應為 ABC；若零 copy 非同步直到完成才釋放，應用不能過早修改。必須寫出是哪一種契約。

spooler 將 job J17 的 3 頁寫入持久 queue 後開始印；printer 已印第 2 頁但 ack 遺失。主機重送整份可能出現重複頁，直接刪 job 可能少印。持久 job id 只能避免主機重複建工作，不能證明不可回復的實體列印 exactly-once；需要裝置本身可查進度/去重契約，否則向使用者回報「結果未知」。

短暫錯誤可重試，但非冪等操作不能盲重送。回傳值通常同時承載長度與失敗資訊；errno 只有在 API 以失敗返回時才有對應意義。controller 的細分 sense/error logs 可供診斷，不可用上一筆殘留 errno 推定本次成功操作失敗。

權限與 kernel data structures 的逐層責任

本例 P 的 fd=4 → per-process fd table → open-file object {offset=8192, readable=true, ops=regular_file_ops} → inode {logical block2→physical block70} → block device/driver queue。兩個 fd 可能共享 open-file offset；另一個獨立 open 不一定共享 offset。

syscall 先驗 fd/mode/range/user pointer；VFS dispatch 按物件型別呼叫 `read`。page cache hit 直接 copy；miss 才進 block/driver。對 socket，ops 指向 socket receive，不能套 inode block70 的流程。名稱解析與 I/O dispatch 是兩階段，不是每次 read 都重新查完整路徑。

I/O port/MMIO/control register 受特權與映射權限保護；user buffer 也須驗證/適當 pin。授權使用某裝置不能推出可 DMA 任意記憶體。資源保留可能造成死結：A 持 scanner 等 printer、B 持 printer 等 scanner；需要一致取得順序/一次申請/回復策略之一。

13.5 From request to hardware

- fd→inode→block→LBA→controller 逐層換算。
- terminal publication 恰一次不代表外部效果 exactly-once。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

從名稱到 controller：不跳過中間映射

固定教學檔案系統：mount `/data` 對應 device (8,2)，`/data/log.bin` 的 inode=37；open 建立 fd=4。read 從 offset=8192 取 4096 B，block size=4096，故 logical block=⌊8192/4096⌋=2、offset-in-block=0；inode 將 logical2 對應 device block70。若裝置 sector=512 B，請求是 LBA=70×8=560、8 sectors。

device (major8,minor2) 選 driver/partition，假設 partition 起始 LBA=2048，實際整碟 LBA=2608。controller queue 的 descriptor 再放入 DMA mapping，不把 fd、inode number 或 user VA 當 LBA。mount/partition/VFS 細節依 OS；以上是完整固定模型，不聲稱所有 UNIX 逐字如此實作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

完成、逾時與取消競爭：誰能釋放 request

狀態為 NEW→QUEUED→IN_FLIGHT→DONE 或 ABORTED；每筆 request 只有一次 terminal publication。IN_FLIGHT 超時先標 timeout 並要求 abort，不可直接視為 ABORTED。只有 hardware quiesced/abort ack 或正常完成，才解除 DMA mapping、釋放 pages。

例 id8：t=0 提交；t=100 timeout；t=101 發 abort；t=102 正常 completion 抵達；t=103 abort 回 already-completed。lock/CAS 保護 terminal 狀態，t=102 贏得唯一完成，t=103 只清理 abort bookkeeping，不再第二次喚醒或解除buffer映射。使用者可能已收到 timeout，但後台仍須保持資源直到確定無 DMA；錯誤通知與資源回收可分離。

一般推理： 只有能把 state 從非 terminal 原子改為 terminal 的分支取得「發布/解除buffer映射一次」資格；所有其他回報看到 terminal 就不再重做。這證明 at-most-once completion，沒有證明外部副作用 exactly-once，也沒有保證壞裝置最終一定回應。

Buffer 與 request control block 的回收時點不同

裝置已停止DMA時，可依完成路徑同步並解除buffer映射；但timer、IRQ、abort callback仍可能存取request的state。故terminal winner不能直接free control block。每個已排入/尚未退出的callback先持有一個穩定reference，退出時release；取消timer若無法證明其callback已退出，也不能提早丟該reference。

延續id8，t=101時有owner、IRQ與abort三個引用，refcount=3。t=102 IRQ取得terminal資格、清DMA buffer後退出，refcount=2；owner交付結果並退出後refcount=1；t=103 abort callback讀到DONE、清自己的bookkeeping後退出，refcount=0，這時才free control block。若另有timer在途，還要再等該引用退出。所有取得reference的入口必須受同一生命期保護；不能先讀裸pointer再對已free物件增ref。

這個reference-count不變量保證「只要任何callback仍可能讀控制狀態，就有正引用維持物件存活」；配合terminal檢查，才能同時避免double completion與use-after-free。若使用request slot/id重用，另需generation/stable event record避免晚到事件誤認成下一筆request。

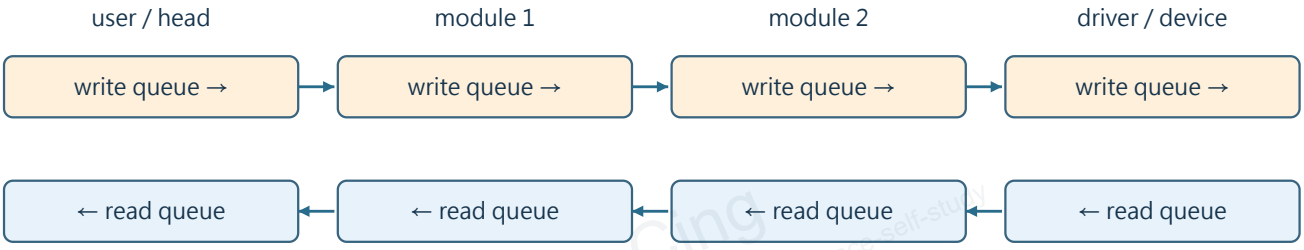
13.6 STREAMS

- 每個元件有 read/write queue；方向相反。
- 以高低水位推導背壓，不假設無限緩衝。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

STREAMS：雙向 queues，不是單列函式呼叫

歷史 STREAMS：每個元件都有兩個方向的 queue



write data 向右；read data 向左。普通資料 queue 滿時，背壓傳向資料的上游。

固定歷史範圍：UNIX System V/Solaris 傳統 STREAMS。stream head 接 user；driver end 接 device；中間可由 ioctl push/pop 模組，每個元件都有 read queue 與 write queue。read data 向 head 上行、write data 向 driver 下行。訊息可含 data/control，不能把 module 一律當成重新 copy payload 的函式。

STREAMS：雙向 queues，不是單列函式呼叫 (續)

put procedure 接到訊息後可立即處理/往下送，也可排入自己的 queue；service procedure 在可運行時取 queue。read/write 是 byte-stream 介面；getmsg/putmsg 可保留訊息控制/資料部分。head 的 user syscall 可 blocking；模組之間的訊息排隊不等於 user thread 逐層同步等待。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

背壓的真數值：容量不會因畫箭頭而增加

教學 write queue Q 容量 1000 B，高水位 800、低水位 300，訊息每筆 200 B。初始 Q=600；接受一筆後 Q=800，設 FULL，上游停止再送。driver 每次取 200：Q 依序 600、400、200；降到低水位以下後 back-enable 上游，才容許排隊服務繼續。這是滯後控制，避免每跨單一門檻就反覆開關。

若上游另存兩筆，停住時只是移到上游 queue，系統總緩衝不是消失；上游也滿才回頭阻塞 head 的 writer。入口若是無法阻塞的網路硬體，buffer 全滿可能 drop，不能在 IRQ 中無限睡等。高優先控制訊息與實際 STREAMS byte accounting 有其規則；本數例只模型化普通固定 200 B data messages，不假稱是所有實作的完整 queue 規格。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

13.7 Performance

- 每 request 的 CPU、memory bytes 與 device service 同時核算。
- Little 關係需相同觀測邊界與有限穩態。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

吞吐瓶頸：CPU、bus 與裝置分別有上限

固定每 request=4 KiB；裝置最多 100000 requests/s；CPU 每筆 5 µs，單核心上限 200000/s；記憶體可供本工作負載 1 GiB/s，每筆總 traffic=12 KiB（DMA 4 KiB，加一次 memory copy 的讀4+寫4），故上限 1 GiB/12 KiB=87381.33/s。整體上限 ≤min(100000,200000,87381.33)=87381.33/s；這是資源界，不保證實際達到。

若真正省掉該 copy，traffic 降到4 KiB，上限升至262144/s，裝置100000/s轉成瓶頸。若「零 copy」仍有其他 memory traffic，須另計。加核心不會突破本例記憶體上限。

batch/coalescing 減少 IRQ 次數，但可能增加排隊延遲。例如 8 筆合併、每筆到達間隔10 µs，首筆需額外等70 µs、末筆0；均勻一批平均35 µs。每IRQ1 µs從每筆1 µs降為每筆0.125 µs，是以延遲換CPU成本。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

延遲、queue depth 與 Little 定律的適用條件

對穩態、有限平均、相同邊界的觀察，L=λW。若完成率80000/s、從進入系統到完成平均W=250 µs，平均在系統 request 數 L=20。W 必須含同一邊界內的 queue+service；不能用平均 queue 長度配總 response time。若裝置 service=100 µs，平均 in-service=8，平均 queue=12，queue wait=150 µs。

最小延遲100 µs與100000/s上限意味維持該吞吐至少需要平均10筆在途；不是queue depth一設10便必定達標。飽和後增加queue常把W拉長，不能無限增加λ。若到達率110000/s而最大完成100000/s、無丟棄/節流，queue持續成長，不能把有限穩態L硬代入。

本頁 Little 關係是本章延伸的流量守恆模型；它既不是 exponential service 假設，也不是 M/M/1 的 W=1/(µ−λ)。沒有額外分布假設不得套後者。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

跨層推導：同一筆讀取的資料與控制

fd4、offset8192→inode37的logical block2→partition LBA560→整碟LBA2608→DMA4096 B。thread blocked，但 request in-flight、device 持 buffer。completion 先確立資料CPU可見、發布狀態，再使 thread ready，最後由scheduler執行返回。

cache hit 可避開最後幾層；nonblocking 可在尚未取得資料時返回 EAGAIN；async 可先返回提交結果。這些變更不會自動消除短讀、錯誤、權限或buffer lifetime。

要判定實作是否正確，沿著「命名/授權→排隊→交付裝置→完成發布→資料使用→資源回收」逐條核對前後條件，而不是由一個成功 callback 推論所有外部效果已持久化。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

代價與可觀察保證

PIO/DMA 是搬運方式，poll/IRQ 是偵測方式，blocking/async 是呼叫契約；它們可以組合，不能互相作同義詞。buffer/cache/spool 解不同層的問題；複本可能提高效能，也可能讓「返回」早於「耐久」。

數值模型需同時交代 bytes/request、CPU time/request、bus bytes/request、service time 與觀測邊界。只有同一分母的成本才能比較；有限批次保留填滿與排空時間，穩態公式明列穩定條件。

本章練習分別驗證握手、遺失喚醒、short I/O、timer、double buffering、取消資源壽命、STREAMS 背壓與瓶頸。詳解保留中間狀態，不把測試少數 trace 當作一般正確性證明。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料

Silberschatz、Galvin、Gagne，*Operating System Concepts*，第九版，第 13 章，印刷頁 587–618；依原書各節議題重新推導，例子、圖及練習為本教材原創。STREAMS 限於書中 System V/Solaris 的歷史模型，並非所有現代 UNIX 的共同實作。Little 定律與 DMA 生命週期分析為明列假設的補充推導。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Abraham Silberschatz, Peter B. Galvin, Greg Gagne. Operating System Concepts, 9th Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study